

DRY-RUN GUIDE | 01\_ascending\_order.java  
Source: java\2nd\_belt\01\_ascending\_order.java

#### CORE BEHAVIOR

Reads N integers and checks adjacent values; equality is allowed. It stops at the first decrease.

#### REPRESENTATIVE INPUT / CALL

Input: N=4; values '1 2 2 5'.

#### STEP TRACE

Every `arr[i]` is `>= arr[i-1]`, so ascending stays true.

#### EXPECTED OUTPUT / RETURN

'Yes'

#### ACTUAL SOURCE EXCERPT

Verbatim lines from the source; long lines wrap. Source files were not modified.

```
001 | /*
002 | Question:
003 | Write a program that takes N integers as input and checks whether the
004 | sequence is in ascending order.
005 |
006 | A sequence is considered ascending if every element is less than or equal
007 | to the next element.
008 |
009 | Input:
010 | The first line contains an integer N.
011 | The second line contains N space-separated integers.
012 |
013 | Output:
014 | Print "Yes" if the sequence is ascending, otherwise print "No".
015 | */
016 |
017 | import java.util.Scanner;
018 |
019 | public class AscendingOrder {
020 |     public static void main(String[] args) {
021 |         Scanner sc = new Scanner(System.in);
022 |
023 |         int n = sc.nextInt();
024 |         int[] arr = new int[n];
025 |
026 |         for (int i = 0; i < n; i++) {
027 |             arr[i] = sc.nextInt();
028 |         }
029 |
030 |         boolean ascending = true;
031 |         for (int i = 1; i < n; i++) {
032 |             if (arr[i] < arr[i - 1]) {
033 |                 ascending = false;
034 |                 break;
035 |             }
036 |         }
037 |     }
```

```
038 |         System.out.println(ascending ? "Yes" : "No");
039 |         sc.close();
040 |     }
041 | }
042 |
```